



南方科技大学

LAB 0: Remote Server & GPU Setup

COE 201 - AI Large Models, Spring 2026

2026-05-17

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

- SSH Connection 1
- Linux Workspace 6
- Dev Environment 9
- GPU Monitoring 12
- 15
- Lab Tasks 16

SSH Connection

SSH (Secure Shell) is the standard for connecting to remote Linux machines.

```
1 ssh <username>@<server-address>  
2 # Example: ssh student@10.20.10.1
```

Shell

Key Concepts:

- **Host:** The IP address or domain name.
- **User:** Your account on the remote machine.
- **Port:** Usually 22 (default).
- **Authentication:** Password (default) or SSH Key (recommended).

SSH Key Authentication (Passwordless Login)



南方科技大学

Stop typing passwords every time!

1. Generate Keys (on your local machine):

```
1 ssh-keygen -t ed25519
```

Shell

2. Copy to Server:

```
1 ssh-copy-id <username>@<server-address>
```

Shell

Now you can log in without a password securely.

On Windows, check this out: <https://www.purdue.edu/science/scienceit/ssh-keys-windows.html> (The OpenSSH client is installed by default on Windows 10/11, but you may still need to enable it manually following the instruction.)

SSH Config for Productivity



南方科技大学

Manage multiple servers easily by editing `~/.ssh/config` on your **local machine**:

```
1 Host coe201-gpu-lab
2     HostName 10.20.10.1
3     User student
4     Port 22
5     IdentityFile ~/.ssh/id_ed25519
```

Text

Usage: Just type `ssh coe201-gpu-lab`!

Pro-tip: VS Code's **Remote-SSH** extension uses this config to give you a full IDE experience on the server.

Copying Files: scp & rsync



How to move your local code or data to the remote server.

Using **scp** (Simple Copy):

```
1 # Copy local file to server
2 scp ./dataset.zip coe201-gpu-lab:~/workspace/
```

Shell

Using **rsync** (Recommended): Only syncs changed files.

```
1 # Sync local folder to remote (note the trailing slashes)
2 rsync -avzP ./my-project/ coe201-gpu-lab:~/workspace/my-project/
```

Shell

- **-a**: Archive mode.
- **-v**: Verbose (shows details).
- **-z**: Compress data during transfer.
- **-P**: Show progress and allow resuming.

Pro-tip: VS Code's file explorer makes this drag-and-drop.

Linux Workspace

Essential Commands



You are in a shell (likely `bash` or `zsh`).

Category	Commands
Navigation	<code>pwd</code> , <code>ls</code> , <code>cd <dir></code>
Files	<code>mkdir</code> , <code>touch</code> , <code>cp</code> , <code>mv</code> , <code>rm</code>
Content	<code>cat</code> , <code>head</code> , <code>tail</code> , <code>grep</code>
Permissions	<code>chmod</code> , <code>chown</code>
Editors	<code>vim</code> , <code>nano</code> , <code>code</code> (VS Code Remote)

Tips: Install `tldr` from <https://github.com/tldr-pages/tldr> and use like this:

- `tldr ls`, `tldr vim`, `tldr rsync`, etc.
- Give you a short and very useful description of commands.

If you close your laptop or the Wi-Fi drops, your SSH connection dies and so does your script.

Solution: Terminal Multiplexers (`tmux`)

- Keeps sessions alive even if you disconnect.
- Allows split panes (left for code, right for monitoring).

Basic Workflow:

1. `tmux new -s training` - Create a session.
2. `Ctrl + b`, then `d` - Detach (the session keeps running).
3. `tmux attach -t training` - Re-attach anytime.

Dev Environment

Modern Python management works great on servers.

Installation:

```
1 curl -LsSf https://astral.sh/uv/install.sh | sh
```

Shell

Project Setup:

1. Clone your repo, with `pyproject.toml`.
2. `uv sync` - Installs everything in a local `.venv`.
3. `uv run python script.py` - Runs your code in the isolated environment.

Conda is a powerful tool for managing both Python environments and non-Python dependencies (like CUDA/CUDNN).

Basic Commands:

- **Create:** `conda create -n coe201 python=3.10`
- **Activate:** `conda activate coe201`
- **List:** `conda env list`
- **Deactivate:** `conda deactivate`
- **Add packages:** `conda install numpy pytorch torchvision`
- **Remove:** `conda env remove -n coe201`

Note: If `conda` is not found, you may need to install **Miniconda** or run `conda init`.

GPU Monitoring

The standard tool for NVIDIA GPUs.

```
1 nvidia-smi
```

Shell

```
ssh -p 16821 root@co ~
root@autodl-container-757d4b9e8c-c6419b51:~# nvidia-smi
Sun Apr 19 18:33:41 2026

+---+
| NVIDIA-SMI 595.58.03                  Driver Version: 595.58.03      CUDA Version: 13.2     |
+---+
| GPU   Name                               Persistence-M   Bus-Id        Disp.A   Volatile Uncorr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap     Bus-Id        Memory-Usage  GPU-Util    Compute M. |
|-----+-----+-----+-----+-----+-----+-----+-----+
| 0     NVIDIA GeForce RTX 5090           On              00000000:5A:00:00 Off      2MiB / 32607MiB | 0%         Default  |
| 42%   26C    P8              19W / 575W           2MiB / 32607MiB | 0%         Default  |
|-----+-----+-----+-----+-----+-----+-----+
|
+---+
| Processes:                               |
|  GPU   GI    CI          PID    Type   Process name          GPU Memory |
|  ID   ID   ID           |
+---+
| No running processes found              |
+---+
root@autodl-container-757d4b9e8c-c6419b51:~#
```

What to watch:

- **GPU-Util:** How busy the compute cores are (0% to 100%).
- **Memory-Usage:** How much VRAM is used (Critical for LLMs!).

Useful when you share a GPU server with others. Check before running your code.

Auto-refresh nvidia-smi:

```
1 watch -n 1 nvidia-smi
```

Shell

nvtop (for a better UI, if installed):

```
1 nvtop
```

Shell

Memory Leak Warning: If your script crashes with “Out of Memory” (OOM), check if old processes are still hanging. Use `kill -9 <PID>` to clear them if necessary.



Lab Tasks

Lab 0 Exercises (Counted into Lab 8 Score)



南方科技大学

Task 1: Server Connectivity [10 points]

- Log in to your assigned GPU server using SSH.
- Configure SSH config (`~/.ssh/config`) for a simple login alias.

Task 2: Environment Checkup (`gpu_check.py`) [10 points]

- Navigate to `labs/lab0`.
- Install dependencies and run `gpu_check.py`.

Task 3: tmux Persistence [10 points]

- Run the check script inside a `tmux` session.
- Practice detaching and re-attaching to maintain the process.



The Missing Semester of Your CS Education: The Shell <https://missing-semester-cn.github.io/2026/course-shell/>

- If you are not familiar with the shell, this is a must-read.

“The Missing Semester” is a practical tools course from MIT — highly recommended reading.

Happy Coding on GPUs!